OSMOD 6 bit / Base 64 Encoding for LB28 Modes

Inventor and Author: Lawrence Byng
Original Publication Date: July 30th 2026

Document Version: 1.1

Primary Components

This document describes the closely related primary components that comprise the character encoding. These are

1. Base 64 / 6 bit character encoding using a subset of 7 bit ASCII codes: The full set of printable ASCII characters is efficiently represented using 6 bits only. Includes data efficient representation of repeated characters using Run Length Encoding.
2. CRC checksums to verify message integrity.: Includes a checksum technique to facilitate message verification.
3. Message padding: Enables use of padding characters to improve decode accuracy.
4. Extendable: Character set selected for compatibility with SAAMFRAM communication protocol to facilitate forms and email.

Bitcodes and Characters

Base Character Set

The characters are detailed in the following table along with the hex code (0x00 to 0x3F) and the 3 bit codes for each the two LB28 carriers ([0,0,0][0,0,0] to [1,1,1][1,1,1]

code	TxRx Character			Bit codes
0x00	<space character>			[0,0,0][0,0,0]
0x01	a			[0,0,0][0,0,1]
0x02	b			[0,0,0][0,1,0]
0x03	c			[0,0,0][0,1,1]
0x04	d			[0,0,0][1,0,0]
0x05	e			[0,0,0][1,0,1]
0x06	f			[0,0,0][1,1,0]
0x07	g			[0,0,0][1,1,1]
0x08	h			[0,0,1][0,0,0]
0x09	i			[0,0,1][0,0,1]
0x0A	j			[0,0,1][0,1,0]
0x0B	k			[0,0,1][0,1,1]
0x0C	l			[0,0,1][1,0,0]
0x0D	m			[0,0,1][1,0,1]
0x0E	n			[0,0,1][1,1,0]
0x0F	o			[0,0,1][1,1,1]
0x10	|	<PAD CHARACTER>	<NON DISPLAY>	[0,1,0][0,0,0]
0x11	p			[0,1,0][0,0,1]
0x12	q			[0,1,0][0,1,0]
0x13	r			[0,1,0][0,1,1]
0x14	s			[0,1,0][1,0,0]
0x15	t			[0,1,0][1,0,1]
0x16	u			[0,1,0][1,1,0]
0x17	v			[0,1,0][1,1,1]
0x18	w			[0,1,1][0,0,0]
0x19	x			[0,1,1][0,0,1]
0x1A	y			[0,1,1][0,1,0]

0x1B	z			[0,1,1][0,1,1]
0x1C	0			[0,1,1][1,0,0]
0x1D	1			[0,1,1][1,0,1]
0x1E	2			[0,1,1][1,1,0]
0x1F	3			[0,1,1][1,1,1]
0x20	4			[1,0,0][0,0,0]
0x21	5			[1,0,0][0,0,1]
0x22	6			[1,0,0][0,1,0]
0x23	7			[1,0,0][0,1,1]
0x24	8			[1,0,0][1,0,0]
0x25	9			[1,0,0][1,0,1]
0x26	[	<SAAMFRAM>	<NON DISPLAY>	[1,0,0][1,1,0]
0x27	]	<SAAMFRAM>	<NON DISPLAY>	[1,0,0][1,1,1]
0x28	(			[1,0,1][0,0,0]
0x29	)			[1,0,1][0,0,1]
0x2A	{	<SAAMFRAM>	<NON DISPLAY>	[1,0,1][0,1,0]
0x2B	}	<SAAMFRAM>	<NON DISPLAY>	[1,0,1][0,1,1]
0x2C	+			[1,0,1][1,0,0]
0x2D	-			[1,0,1][1,0,1]
0x2E	:			[1,0,1][1,1,0]
0x2F	;			[1,0,1][1,1,1]
0x30	=	<Escape Char #2>	<NON DISPLAY>	[1,1,0][0,0,0]
0x31	?	<SAAMFRAM>	<NON DISPLAY>	[1,1,0][0,0,1]
0x32	@			[1,1,0][0,1,0]
0x33	^	<SAAMFRAM>	<NON DISPLAY>	[1,1,0][0,1,1]
0x34	,			[1,1,0][1,0,0]
0x35	‘			[1,1,0][1,0,1]
0x36	.			[1,1,0][1,1,0]
0x37	/	<Escape Char #1>	<NON DISPLAY>	[1,1,0][1,1,1]
0x38	~	<SAAMFRAM>	<NON DISPLAY>	[1,1,1][0,0,0]
0x39	_			[1,1,1][0,0,1]
0x3A	%			[1,1,1][0,1,0]
0x3B	!			[1,1,1][0,1,1]
0x3C	#			[1,1,1][1,0,0]
0x3D	$			[1,1,1][1,0,1]
0x3E	&			[1,1,1][1,1,0]

0x3F	*			[1,1,1][1,1,1]

‘=’ Escape Seq

Display Character	‘=’ Escape Seq.	Control Action		
=	==			
[	=a			
]	=b			
{	=c			
}	=d			
	=e	<NOT USED>		
~	=f			
|	=g			
	=h	Next word in CAPS		
	=i	CAPS lock on		
	=j	Bold mode on		
	=k	Italics mode on		
	=l	Underscore mode on		
	=m	Revert normal mode		
	=n	<New Line / CR LF>		
	=o	<NOT USED>		
>	=p			
?	=q			
<	=r			
“	=s			
\	=t			
^	=u			
`	=v			
	=w	<BOS>		
	=x	<EOS>		
	=y	<TAB>		
	=z	<EOM>		

ASCII Xref + Escape Sequences

ASCII Character	ASCII code	LB64 Code	Escape Seq	notes
<space>	32	0x00		
!	33	0x3B		
“	34		=s	
#	35	0x3C		
$	36	0x3D		
%	37	0x3A		
&	38	0x3E		
‘	39	0x35		Single quote
(	40	0x28		
)	41	0x29		
*	42	0x3F		
+	43	0x2C		
,	44	0x34		
-	45	0x2D		
.	46	0x36		
/	47		//	
0	48	0x1C		
1	49	0x1D		
2	50	0x1E		
3	51	0x1F		
4	52	0x20		
5	53	0x21		
6	54	0x22		
7	55	0x23		
8	56	0x24		
9	57	0x25		
:	58	0x2E		
;	59	0x2F		
<	60		=r	
=	61		==	
>	62		=p	
?	63		=q	

@	64	0x32		
A	65		/a	
B	66		/b	
C	67		/c	
D	68		/d	
E	69		/e	
F	70		/f	
G	71		/g	
H	72		/h	
I	73		/i	
J	74		/j	
K	75		/k	
L	76		/l	
M	77		/m	
N	78		/n	
O	79		/o	
P	80		/p	
Q	81		/q	
R	82		/r	
S	83		/s	
T	84		/t	
U	85		/u	
V	86		/v	
W	87		/w	
X	88		/x	
Y	89		/y	
Z	90		/z	
[	91		=a	
\	92		=t	
]	93		=b	
^	94		=u	
_	95	0x39		
`	96		=v	Grave accent
a	97	0x01		
b	98	0x02		
c	99	0x03		

d	100	0x04		
e	101	0x05		
f	102	0x06		
g	103	0x07		
h	104	0x08		
i	105	0x09		
j	106	0x0A		
k	107	0x0B		
l	108	0x0C		
m	109	0x0D		
n	110	0x0E		
o	111	0x0F		
p	112	0x11		
q	113	0x12		
r	114	0x13		
s	115	0x14		
t	116	0x15		
u	117	0x16		
v	118	0x17		
w	119	0x18		
x	120	0x19		
y	121	0x1A		
z	122	0x1B		
{	123		=c	
|	124		=g	
}	125		=d	
~	126		=f	

Example

```
message: WH6GGO: The Queen Of Hearts SHE MADE SOME TARTS all on a summer's day. The Knave Of Hearts HE STOLE THE TARTS and took them clean away. The King Of Hearts CALLED FOR THE TARTS and beat the knave full sore. The Knave Of Hearts BROUGTH BACK THE TARTS and vowed he'd steal no more.

translated: =iwh6ggo: t=mhe /queen /of /hearts =ishe made some tarts =mall on a summer's day. /the /knave /of /hearts =ihe stole the tarts =mand took them clean away. /the /king /of /hearts =icalled for the tarts =mand beat the knave full sore. /the /knave /of /hearts =ibrougth back the tarts =mand vowed he'd steal no more.
```

Run Length Encoding

RLE utilizes the character / followed by one or more integers followed by a character. Can only be used for representing non-numeric sequences of repeating characters.

For example

‘this is a looooooooooooong word’ is represented as:

‘this is a l/12o word’

Padding Character

A padding character of | is used to extend the message length out to a fixed number of characters. This increases decode accuracy on two counts:- 1) lengthening the message to provide more decode points and 2) using a character code that increases the message decode accuracy (bit code [0,1,0] [0,0,0]

CRC checksum

Attribution - CRC Polynomials - Philip Koopman - Carnegie Mellon University

The general purpose polynomials used are derived from the Best CRC Polynomials document by Philip Koopman, Carnegie Mellon University -

https://users.ece.cmu.edu/~koopman/crc/

Each message is fragmented into chunks of 16 characters. The start of each chunk begins with the | character followed by a numeric alpha numeric code from 0 to 9 and a to z (36 segments maximum). The end of the fragment contain a two digit checksum. The last fragment also has a | character on the end.

Here is the earlier example fragmented into sections along with the CRC checksums:

```
protected_string: |0=iwh6ggo: t=mhe ss|1/queen /of /hear20|2ts =ishe
made sors|3me tarts =mall out|4n a summer's day8h|5. /the /knave /ok6|6f
/hearts =ihe si7|7tole the tarts =10|8mand took them cfh|9lean away.
/the fb|a/king /of /heart7l|bs =icalled for tj5|ch00|||
```

```
The sections a sequenced from |0 to |c
```

```
The message ends with 3 padding characters |||
```

Pre-Defined Messages

Beacon General:	<CALLSIGN>: BG(<GRID LOCATOR>)
Beacon Net:	<CALLSIGN>: BN(<GRID LOCATOR>, <Time UTC>, <Frequency>, <Mode>)
Beacon Text Msg:	<CALLSIGN>: BT(<GRID LOCATOR>, <Text Msg>)
Beacon CQCQ:	<CALLSIGN>: BC(<GRID LOCATOR>, <Frequency>, <Mode>)
Beacon Alert (Red):	<CALLSIGN>: BR(<GRID LOCATOR>, <Message>)
Beacon Alert (Green):	<CALLSIGN>: BA(<GRID LOCATOR>, <Message>)